



Introduction to NLP

AIAA 4051 Lecture 11

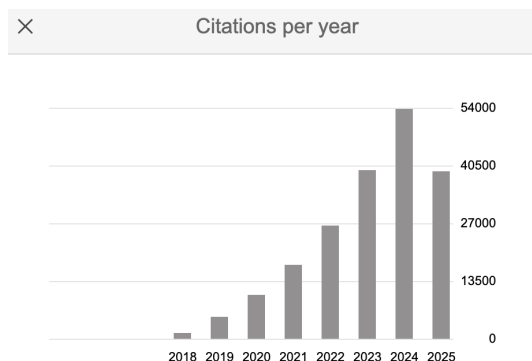
Instructor: Sihong Xie

Email: sihongxie@hkust-gz.edu.cn

AI Thrust, HKUST(GZ)

Transformer

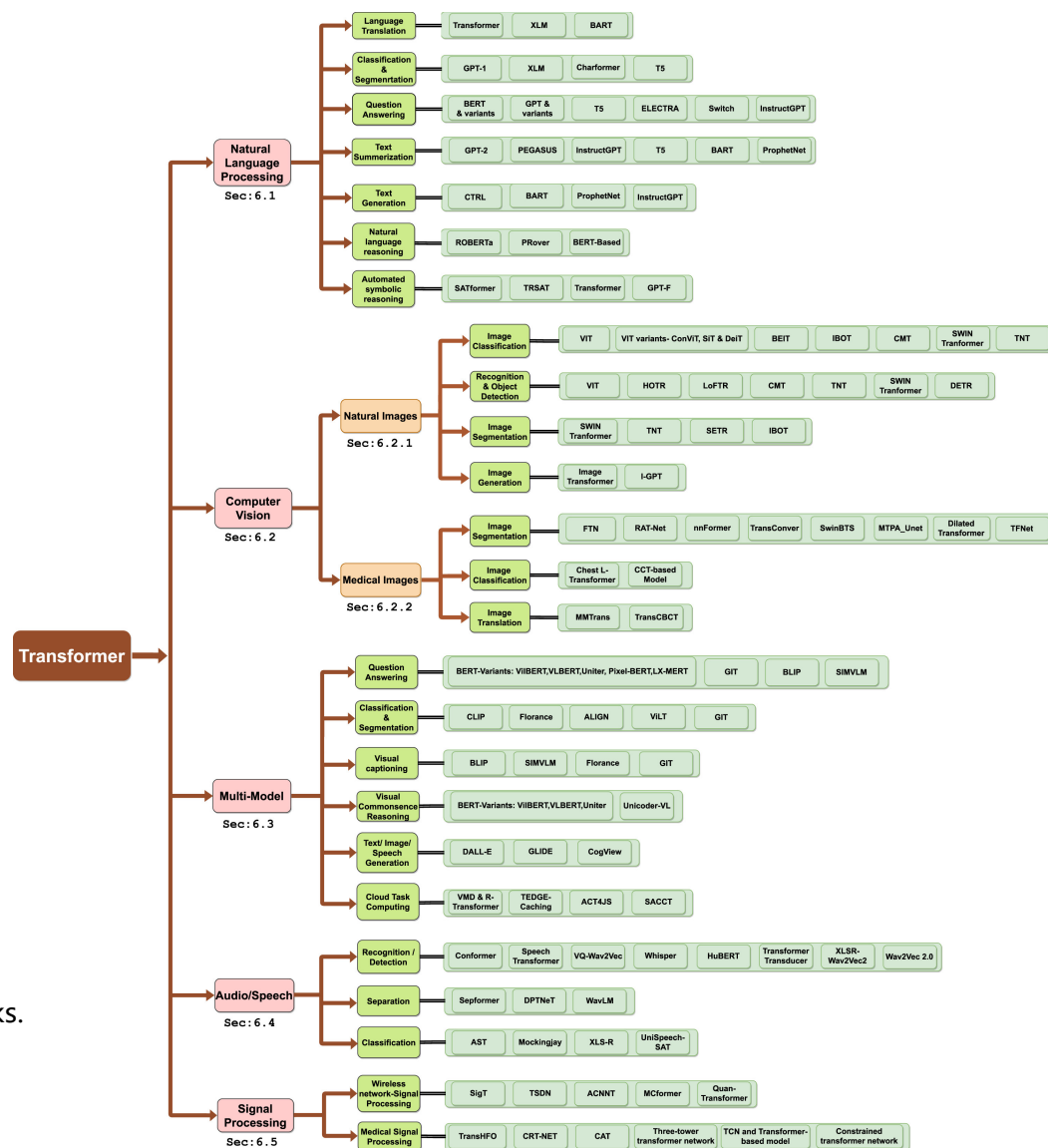
- Started from the 2017 paper "Attention is all you need".



- Extended to computer vision and robotics and data science.
- The current popular GPT-like applications are based on Transformer.

Image Source:

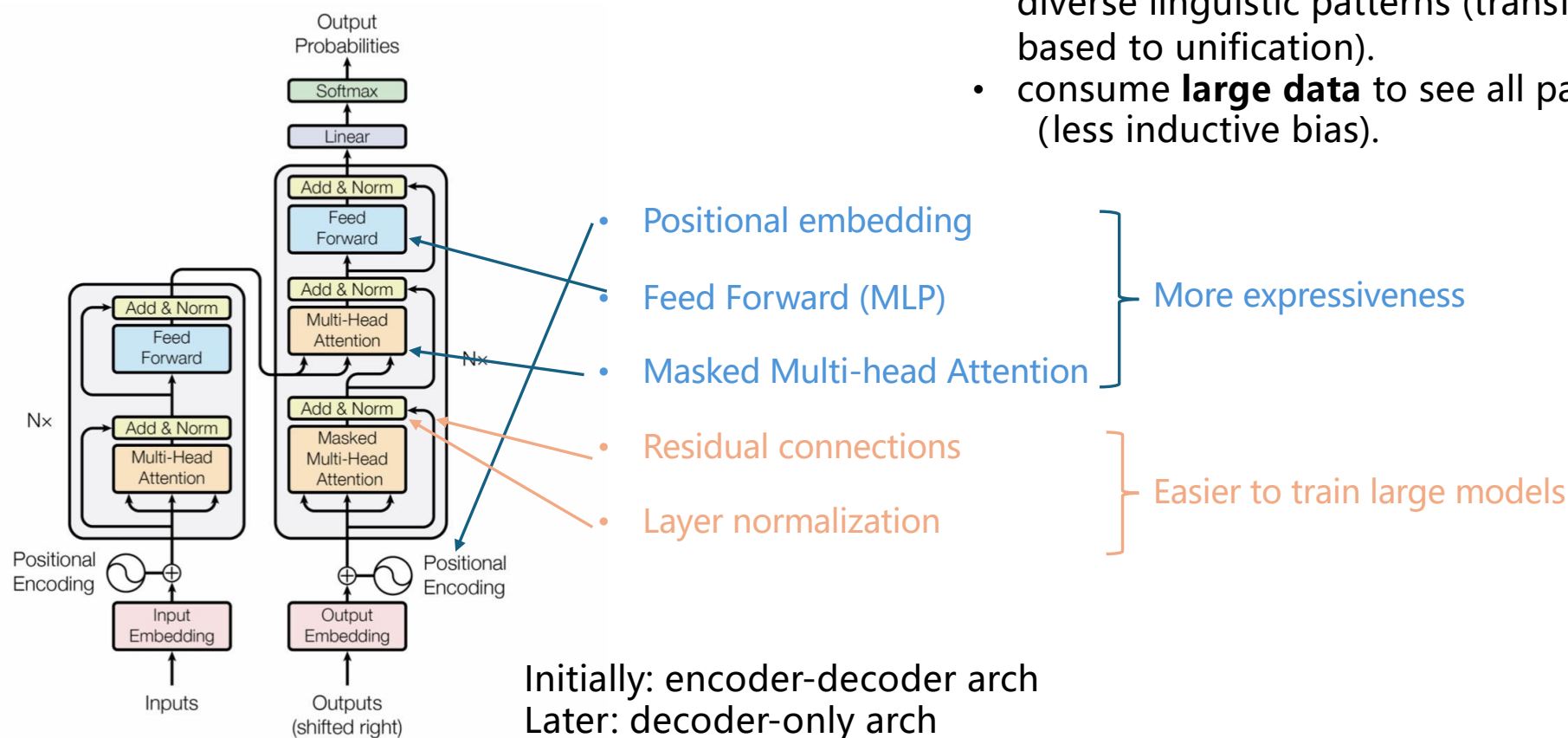
A comprehensive survey on applications of transformers for deep learning tasks.
Saidul Islam, etc. 2024



Transformer

- A breakdown of the transformer

- As a foundation model, it needs to
- be a **large model** to model general and diverse linguistic patterns (transformer-based to unification).
 - consume **large data** to see all patterns (less inductive bias).



Transformer: positional embedding

- Positional embedding

- To give a positional identification to a token.
- The position of a token is important in deriving linguistic features.

Trust is what builds trust Tokenization {trust, trust, is, what, builds}

embedding("trust")=embedding("trust")

- But they are different:

- **Syntactically**: the first "trust" is subject, while the second is a object.
- **Semantically**: the first "trust" the thing that **creates something**, while the second is the thing **being created**.

- Another example

A man bites a dog Tokenization {a, man bites, dog}

vs.

A dog bites a man

Transformer: positional embedding

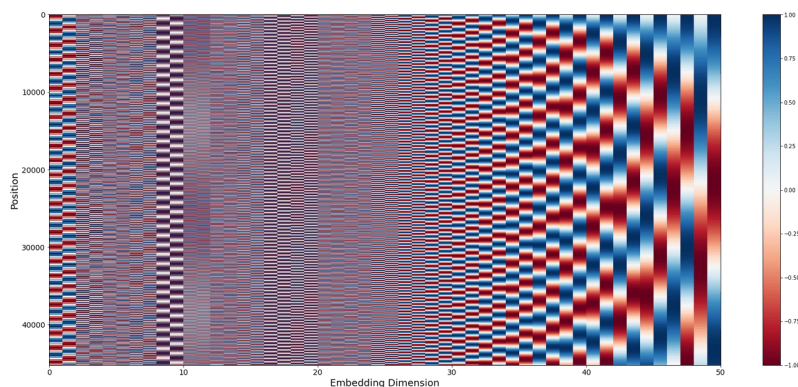
- Positional embedding

Trust is what builds trust

0 1 2 3 4

`embedding("trust") + position_vector(0)`
!=
`embedding("trust") + position_vector(4)`

- Positional encoding (fixed)



$$PE_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$

pos: position of a token
i: the i -th dimension of the embedding
d: the total number of dimensions of the embedding.

Transformer: positional embedding

- A running example

Example: sentence = ["The", "cat", "sat"]

"The" (E_{The}): [0.1, 0.2, 0.3, 0.4]

"cat" (E_{cat}): [0.5, 0.6, 0.7, 0.8]

$$PE_{(0,0)} = \sin\left(\frac{0}{10000^{0/4}}\right) = \sin(0) = 0$$

$$PE_{(1,0)} = \sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1) \approx 0.8415$$

$$PE_{(2,0)} = \sin\left(\frac{2}{10000^{0/4}}\right) = \sin(2) \approx 0.9093$$

$$PE_{(0,1)} = \cos\left(\frac{0}{10000^{0/4}}\right) = \cos(0) = 1$$

$$PE_{(1,1)} = \cos\left(\frac{1}{10000^{0/4}}\right) = \cos(1) \approx 0.5403$$

$$PE_{(2,1)} = \cos\left(\frac{2}{10000^{0/4}}\right) = \cos(2) \approx -0.4161$$

$$PE_{(0,2)} = \sin\left(\frac{0}{10000^{2/4}}\right) = \sin(0) = 0$$

$$PE_{(1,2)} = \sin\left(\frac{1}{10000^{2/4}}\right) \approx \sin(0.01) \approx 0.01$$

$$PE_{(2,2)} = \sin\left(\frac{2}{10000^{2/4}}\right) \approx \sin(0.02) \approx 0.02$$

$$PE_{(0,3)} = \cos\left(\frac{0}{10000^{2/4}}\right) = \cos(0) = 1$$

$$PE_{(1,3)} = \cos\left(\frac{1}{10000^{2/4}}\right) \approx \cos(0.01) \approx 0.99995$$

$$PE_{(2,3)} = \cos\left(\frac{2}{10000^{2/4}}\right) \approx \cos(0.02) \approx 0.9998$$

$$H_{\text{The}} = E_{\text{The}} + PE_{(0)}$$

$$H_{\text{cat}} = E_{\text{cat}} + PE_{(1)}$$

$$H_{\text{sat}} = E_{\text{sat}} + PE_{(2)}$$

$$H_{\text{The}} = [0.1, 0.2, 0.3, 0.4] + [0, 1, 0, 1]$$

$$H_{\text{cat}} = [0.5, 0.6, 0.7, 0.8] + [0.8415, 0.5403, 0.01, 0.99995]$$

$$H_{\text{sat}} = [0.9, 1.0, 1.1, 1.2] + [0.9093, -0.4161, 0.02, 0.9998]$$

$$H_{\text{The}} = [0.1, 1.2, 0.3, 1.4]$$

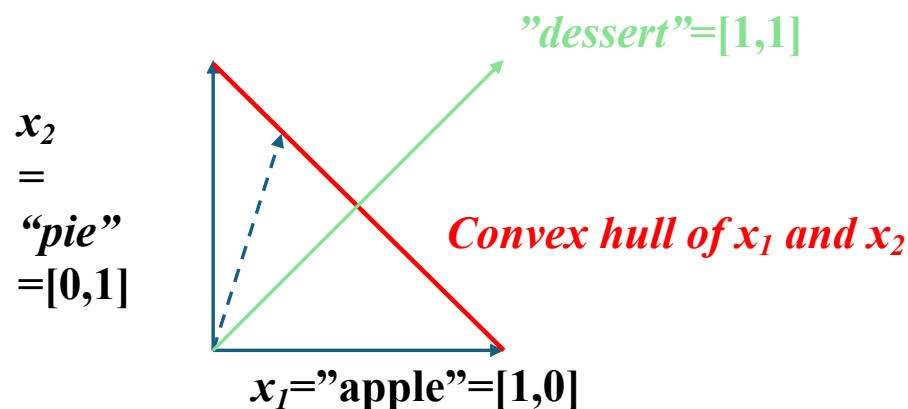
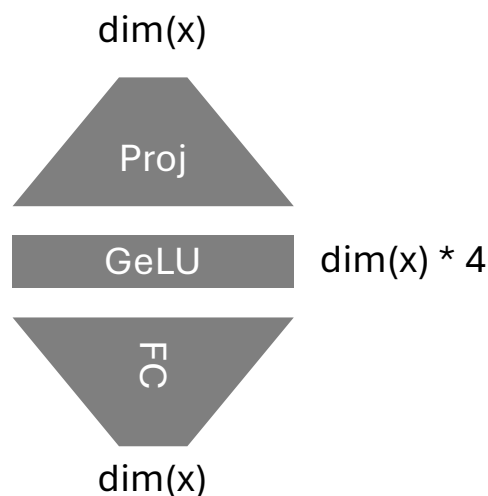
$$H_{\text{cat}} = [1.3415, 1.1403, 0.71, 1.79995]$$

$$H_{\text{sat}} = [1.8093, 0.5839, 1.12, 2.1998]$$

Transformer: MLP layer

- MLP (multi-layered perception) Feed Forward Layer
 - MLP first maps x to higher dimensional space, then takes the GeLU nonlinear mapping, and finally projects back to $\text{dim}(x)$.
- Why it is necessary?
 - The self-attention layer can only find linear combination of value vectors.
 - What if some non-linearity is needed?

It is important that the input and output have the same dimensionality, since any number of transformer blocks can be appended.



Transformer: attention

- The central of Transformer: attention mechanism
 - To address the gradient propagation issues of RNN/LSTM.

"I grew up in **France**, where I spent many years surrounded by my family and friends, so I speak fluent **French**."

Long-range dependencies

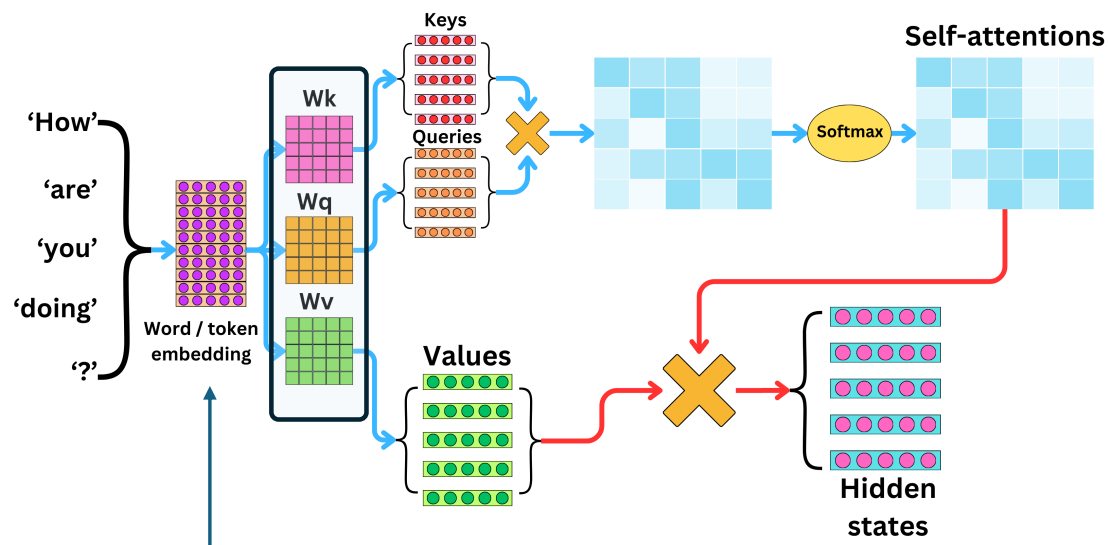
- Why it is hard to learn the dependency?
 - If the model makes a wrong prediction "**English**" at the end, it is possible that the training data have more frequent "**fluent English**" occurrences (short-range dep).
 - The model forward passes are more influenced by the recent word such as "fluent", while the far-away word "France" has its semantic vector modified by all the other words in the middle and the many multiplication of the weight matrix of RNN.

$$\begin{array}{lcl} h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t) & \text{rollout} & h_t = \tanh(W_{hh}\tanh(W_{hh}h_{t-2} + w_{xh}x_{t-2}) + w_{xh}x_{t-1}) \\ y_t = W_{hy}h_t & \longrightarrow & \end{array}$$

Transformer: attention

```
# calculate query, key, values for all heads in batch and move head forward to be the batch dim
q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
```

$\text{self.c_attn}=[W_k, W_q, W_v], \text{shape}=(n_embd, 3*n_embd)$



$x.\text{shape}=[\text{Batch_size}, \text{Token_len}, n_embd]$

$C = n_h * h_s = n_embd$

$k.\text{shape}[B, T, n_h * h_s]$

view



$k.\text{shape}[B, T, n_h, h_s]$

transpose



$k.\text{shape}[B, n_h, T, h_s]$

This is a 5x5 matrix as the input has $T=5$ tokens. The size of the attention matrix scales $O(T^2)$

Note: hidden state has $h_s = 5$ and is just one of the heads' size. For a multiple head attention, there are n_h heads, so that the total dimension is $n_h * h_s$.

Transformer: attention

```
att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float('-inf'))
att = F.softmax(att, dim=-1)
```

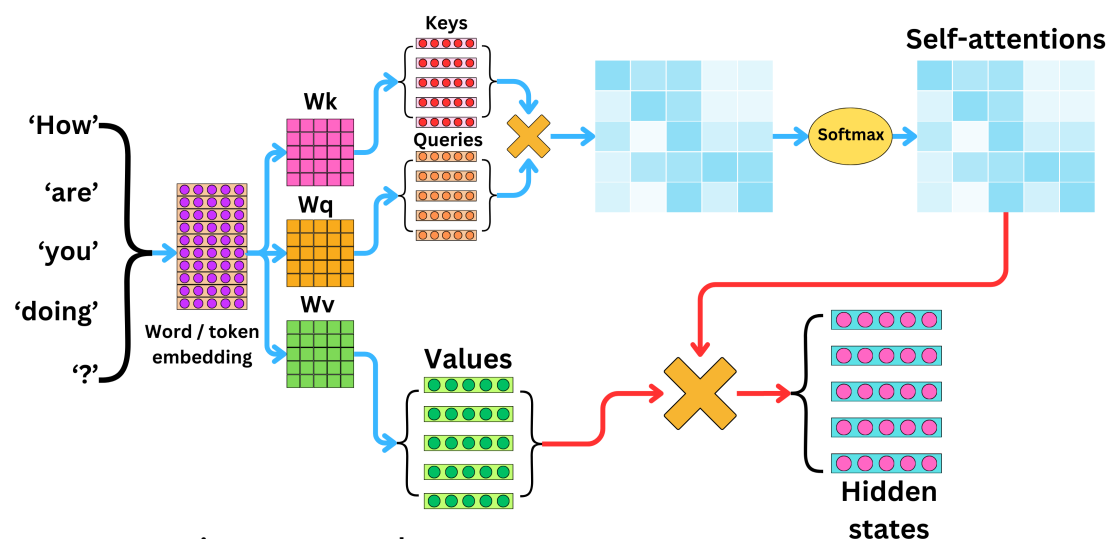
To prevent the inner products from inflating the scales.

$q.shape = [B, nh, T, hs]$

$k.transpose(-2, -1).shape = [B, nh, hs, T]$

$(q @ k).shape = [B, nh, T, T]$

$self.bias[:, :, :T, :T]$ has the upper triangle = 0



```
>>> import torch
>>> a=torch.tensor([1,2,3,float('-inf'), float('-inf'), float('-inf')])
>>> F.softmax(a)
tensor([0.0900, 0.2447, 0.6652, 0.0000, 0.0000, 0.0000])
```

Causal Attention: don't look into the future

Attention weight between the tokens corresponding to "Life" and "short"

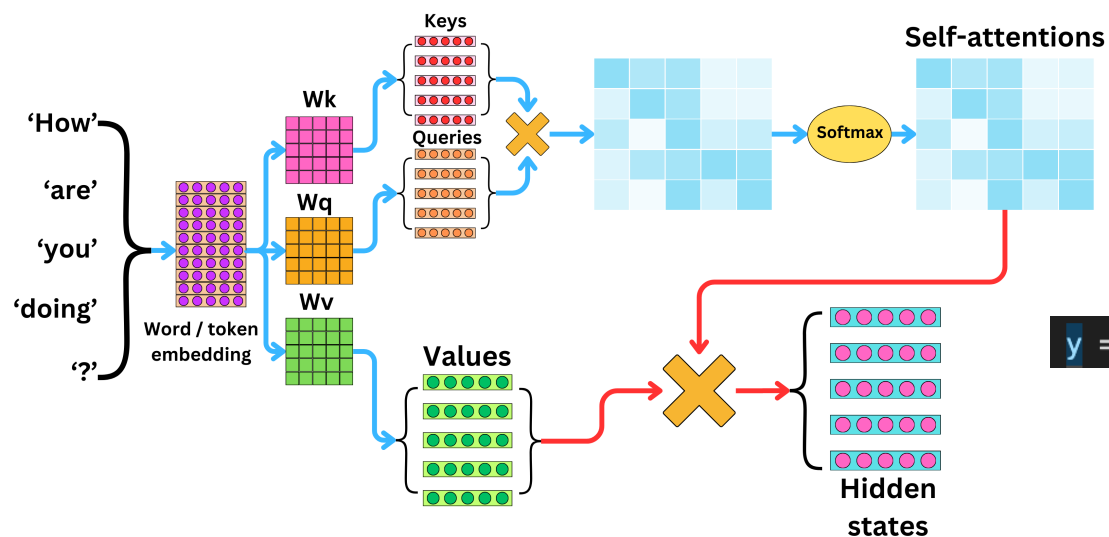
In the row for corresponding to "Life" mask out all words that come after "Li"

	Life	is	short	eat	desert	first
Life	0.17	0.13	0.18	0.16	0.15	0.18
is	0.03	0.68	0.02	0.08	0.14	0.02
short	0.19	0.06	0.25	0.14	0.11	0.23
eat	0.15	0.21	0.14	0.16	0.17	0.14
desert	0.13	0.27	0.11	0.16	0.18	0.12
first	0.19	0.02	0.31	0.11	0.07	0.27

Attention weights calculated in the previous "Self-Attention" section

	Life	is	short	eat	desert	first
Life	0.17	0.13	0.18	0.16	0.15	0.18
is	0.03	0.68	0.02	0.08	0.14	0.02
short	0.19	0.06	0.25	0.14	0.11	0.23
eat	0.15	0.21	0.14	0.16	0.17	0.14
desert	0.13	0.27	0.11	0.16	0.18	0.12
first	0.19	0.02	0.31	0.11	0.07	0.27

Transformer: attention



$\text{att.shape}=[B, \text{nh}, T, T]$

$\text{v.shape}=[B, \text{nh}, T, \text{hs}]$

$(\text{att} @ \text{v}).\text{shape}=[B, \text{nh}, T, \text{hs}]$

`y = att @ v # (B, nh, T, T) x (B, nh, T, hs) -> (B, nh, T, hs)`

`y = y.transpose(1, 2).contiguous().view(B, T, C) # re-assemble all head outputs side by side`



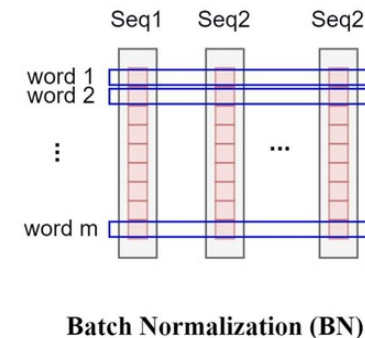
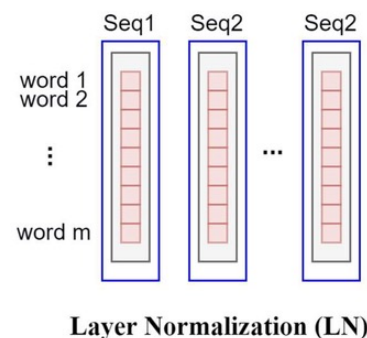
Transformer: layer normalization

$$\text{layer-norm}(x) = \frac{x - \text{mean}(x)}{\sqrt{\text{variance}(x) + \epsilon}} \cdot \gamma + \beta$$

Computed over all values within each input sequence

Learnable parameters

Small additive constant



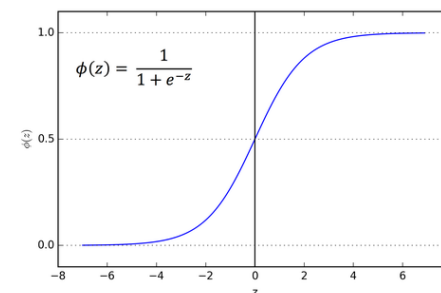
• Intuitions:

- For all neurons at the same layer, the “scale” of their activations should remain about the same as the input.
- After going through many layers, the scale of activations should remain about the same.

• Why it is necessary?

- If several layers enlarge or shrink the activation scales dramatically, there is the so-called “saturation of softmax” and consequently the “vanishing gradient” issue.

```
>>> a=torch.tensor([1.,2.,3.])
>>> b=torch.tensor([10.,20.,30.])
>>> softmax(a)
tensor([0.0900, 0.2447, 0.6652])
>>> softmax(b)
tensor([2.0611e-09, 4.5398e-05, 9.9995e-01])
```



Transformer: layer normalization

1 batch with 3 samples

	r1	r2	r3
c1	2	5	7
c2	6	4	2
c3	8	1	3
c4	9	7	2
c5	3	8	1
Mean	5.6	5	3
Std_dev	2.73	2.45	2.1

Features ↑

↓ ↓ ↓

- Ignoring the learnable parameters, the first sample's features are normalized to

- $(2-5.6)/2.73=-1.3186813187$

- $(6-5.6)/2.73=0.1465201465$

- $(8-5.6)/2.73=0.8791208791$

- $(9-5.6)/2.73=1.2454212454$

- $(3-5.6)/2.73=-0.9523809524$

The differences between two activations are much small.

Image source:

https://miro.medium.com/v2/resize:fit:1400/1*F4pVp_fr90NDdWhKwcTBLQ.png

Transformer: residual connections

- Residual connections
 - $x = \text{MHA}(x) + x$ and $x = \text{MLP}(x) + x$
- Intuitions:
 - At any layer, if a mapping is unnecessary for a token, just let that mapping to be zero and push the input to the next layer.
 - Mapping from $[9, 11, 19, 21]$ to $[10, 12, 18, 20]$ is hard, but from $[9, 11, 19, 21]$ to the residual $[-1, -1, 1, 1]$ is much easier.
- Why it is necessary?
 - Allow information to directly forward to the next level without MHA or MLP (which are just *learned* to be the zero mapping).
 - Allow gradients (of loss function of supervision) to directly backprop to the earlier layers, without too many multiplications and eventual gradient explosion or vanishing.

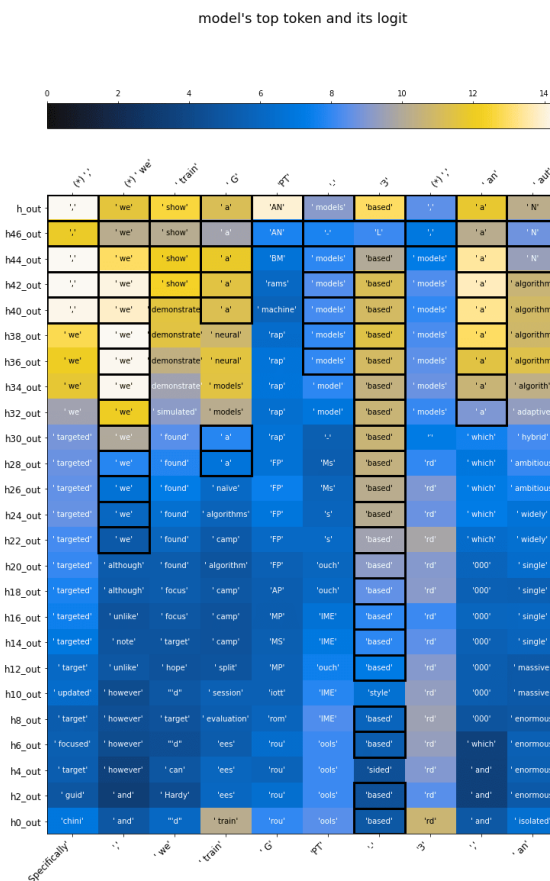
Transformer: residual connections

- Another viewpoint of transformer: residual-centered $x_{n+1} = x_n + \text{Sublayer}(x_n)$
- Layers as "Add-Only" Modules
 - the Attention and MLP layers aren't *reforming* the representation; they are **reading** from the stream, performing a small calculation, and **writing** the result back to it.
 - **Attention layers** move information from one token's position in the stream to another, thus connecting far-away information.
 - **MLP layers** process the information at a single position, often acting like key-value memories to refine the data (the "Processing").
- Linear Representation
 - Because the updates are additive, the residual stream remains surprisingly **linear**. You can think of the final output of transformer as the sum of the original embedding plus a series of "*delta*" updates:

$$x_{final} = x_{embed} + \Delta_{atten_1} + \Delta_{mlp_1} + \dots + \Delta_{atten_L} + \Delta_{mlp_L}$$

Transformers: mechanistic interpretability

- Since the stream is additive, take the hidden state at *any* intermediate layer and pass it through the final Unembedding layer to see what the model "thinks" the answer is at that exact moment.
- The model's prediction evolve from a simple bigram guess in layer 2 to the correct complex fact by layer 12.



In the figure, each position at each layer is the most likely word from unembedding.

The color means the saturation of the logits: Blue means less certainty, while yellow means more certainty of the predicted token.

Image source:

https://res.cloudinary.com/lesswrong-2-0/image/upload/f_auto,q_auto/v1/mirroredImages/AckRB8wDpdaN6v6ru/ccfmt4rt3aegjfi7lo8

Transformer: mechanistic interpretability

- Information flow: how information flow from bottom to top to predict the next token.

"When Mary and John went to the store, John gave a book to"

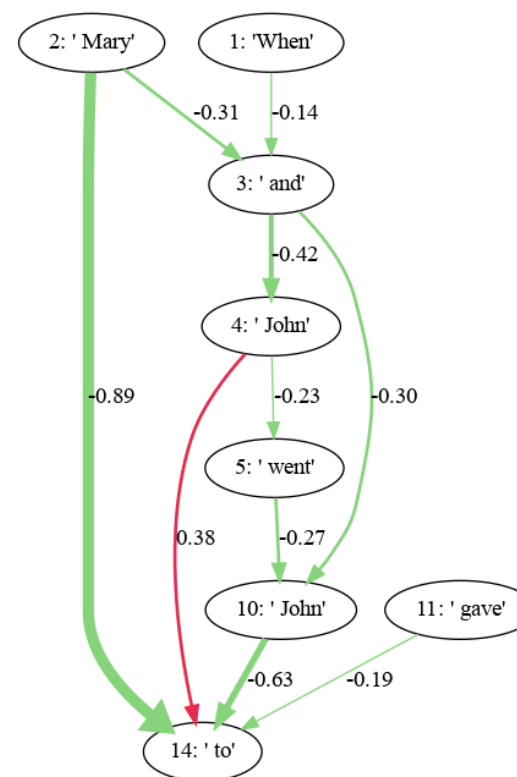


Image source:

<https://www.lesswrong.com/posts/xNgdJEep9DQQWhSbv/understanding-the-information-flow-inside-large-language>

Transformer: training

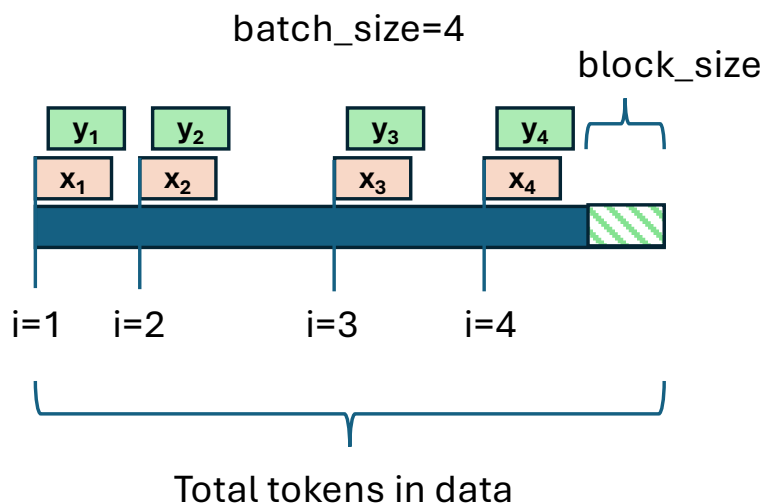
- Training

- Prepare training and validation data
 - Load raw texts.
 - Use an existing encoder to turn text token into indices.
 - Save to binary files.

```
with open(input_file_path, 'r', encoding='utf-8') as f:
    data = f.read()
n = len(data)
train_data = data[:int(n*0.9)]
val_data = data[int(n*0.9):]

# encode with tiktoken gpt2 bpe
enc = tiktoken.get_encoding("gpt2")
train_ids = enc.encode_ordinary(train_data)
val_ids = enc.encode_ordinary(val_data)
```

- Sample a batch: randomly



Load part (not all) of the file to memory.

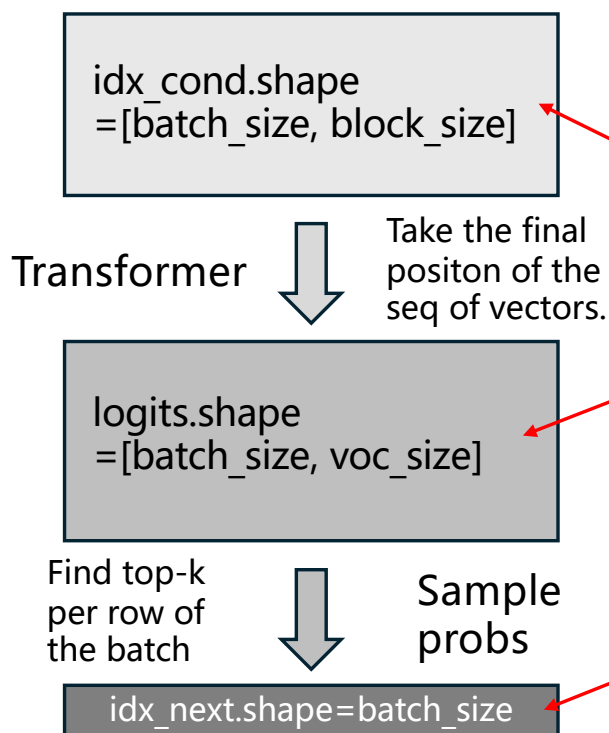
```
# poor man's data loader
data_dir = os.path.join('data', dataset)
def get_batch(split):
    # We recreate np.memmap every batch to avoid a memory leak, as per
    # https://stackoverflow.com/questions/45132940/numpy-memmap-memory-usage-want-to-iterate-once/61472122#61472122
    if split == 'train':
        data = np.memmap(os.path.join(data_dir, 'train.bin'), dtype=np.uint16, mode='r')
    else:
        data = np.memmap(os.path.join(data_dir, 'val.bin'), dtype=np.uint16, mode='r')
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([torch.from_numpy((data[i:i+block_size]).astype(np.int64)) for i in ix])
    y = torch.stack([torch.from_numpy((data[i+1:i+1+block_size]).astype(np.int64)) for i in ix])
    if device_type == 'cuda':
        # pin arrays x,y, which allows us to move them to GPU asynchronously (non_blocking=True)
        x, y = x.pin_memory().to(device, non_blocking=True), y.pin_memory().to(device, non_blocking=True)
    else:
        x, y = x.to(device), y.to(device)
    return x, y
```

next token prediction

Transformer: inference

INEFFICIENCY: 1) need a for loop to generate token-by-token;
2) for each token, all previous tokens should be forwarded.

- Inference



```
@torch.no_grad()
def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):
    """
    Take a conditioning sequence of indices idx (LongTensor of shape (b,t)) and complete
    the sequence max_new_tokens times, feeding the predictions back into the model each time.
    Most likely you'll want to make sure to be in model.eval() mode of operation for this.
    """
    for _ in range(max_new_tokens):
        # if the sequence context is growing too long we must crop it at block_size
        idx_cond = idx if idx.size(1) <= self.config.block_size else idx[:, -self.config.block_size:]
        # forward the model to get the logits for the index in the sequence
        logits, _ = self(idx_cond)
        # pluck the logits at the final step and scale by desired temperature
        logits = logits[:, -1, :] / temperature
        # optionally crop the logits to only the top k options
        if top_k is not None:
            v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
            logits[logits < v[:, [-1]]] = -float('Inf')
        # apply softmax to convert logits to (normalized) probabilities
        probs = F.softmax(logits, dim=-1)
        # sample from the distribution
        idx_next = torch.multinomial(probs, num_samples=1)
        # append sampled index to the running sequence and continue
        idx = torch.cat((idx, idx_next), dim=1)

    return idx
```

Umembedding is performed inside the Transformer.

Research project 1

- We released the first project option on Canvas.
- Mechanistic interpretability of Transformer (GPT-2)
 - Small enough to fine-tune and analyze on mid-end GPUs.
- Main tasks:
 - Run the LRP algorithm to show contributions of input and parameters to predicted tokens.
 - Compare the contributions before and after fine-tuning.
 - Research extension: how layer normalization impact the attribution? How to discover input and parameter conflicts?
- Top teams will be featured on the course promotion materials, and supported for paper publications.

Extra readings

- For a very good visualization of transformer, see the Youtube video
 - <https://www.youtube.com/watch?v=9-Jl0dxWQs8>
- For mechanistic interpretability of LLM, see the blog
 - <https://www.lesswrong.com>
- For a very good implementation from scratches, see the following:
 - <https://github.com/karpathy/nanoGPT> (by Andrej Karpathy)
 - <https://www.youtube.com/watch?v=kCc8FmEb1nY> (explaining the above codes).

| Demon

- Nothing is better than Karpathy's nanoGPT, just go there and read the codes and run them on your own computer for a pre-training.

Conclusion

- Transformers are now everywhere for many tasks in NLP, CV, and robotics.
- Diving deep into a Transformer.
- Mechanistic interpretability of Transformers.

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- Q1 (T/F): layer normalization is to make model training more stable.
 - Q2 (T/F): attention mechanism scales linearly as the length of the sequence.
 - Q3 (short answer): write down the equation to calculate attention matrix using Q, K, and V matrices.
 - Q4 (multiple choices): which of the following is not part of the architecture of a transformer? A) attention; B) MLP layer; C) interpretability; D) positional embedding.
 - A: T, F, $\text{softmax}(QK^T/\text{sqrt}(d))$, C